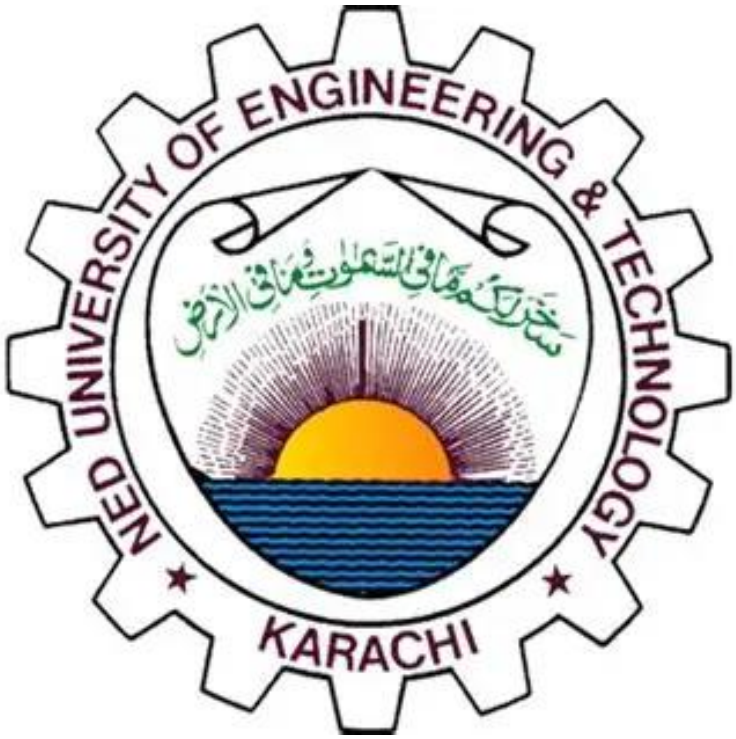


NED UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Department of Computer Science & Information Technology

Bachelor of Science in Computer Science (Specialization in Cyber Security)

COMPLEX COMPUTING ACTIVITY (CCA) REPORT

Spam-Shield

AI-Based SMS Spam and Scam Message Detection System

Course Details

Course Code and Title	CT-264 – Programming for Artificial Intelligence
Assessment Type	Complex Computing Activity (CCA)
Course Learning Outcome	CLO 3 (C3) – Identify appropriate AI techniques to solve classification problems of moderate complexity
Project Title	SpamShield: AI-Based SMS Spam and Scam Message Detection System

Group Members

Roll Number	Student Name	Primary Role & Contribution
CT-24278	Muhammad Saad	Exploratory Data Analysis (EDA), Statistical Visualization & Imbalance Analysis
CT-24125	Haris Zamir	Machine Learning Model Implementation, Hyperparameter Tuning & Metric Evaluation
CT-24136	Ali Hassan	Streamlit Web Interface Development, Risk Score Calibration & Technical Documentation
CT-24138	Muhammad Raza	Core Python Pipeline Architecture, Regex Text Preprocessing & TF-IDF Vectorization

Table of Contents

Abstract 3

1. Introduction & Problem Identification 3
- 1.1 Background and Context 3
- 1.2 Problem Statement 3
- 1.3 Objectives 4
- 1.4 Real-World Significance & Cyber Security Relevance 4

2. Dataset Description & Exploratory Data Analysis 4
- 2.1 Data Source and Structure 4
- 2.2 Data Cleaning and Deduplication 4
- 2.3 Class Imbalance Analysis 5
- 2.4 Statistical Summary of Structural Features 5

3. Methodology & System Architecture 7
- 3.1 System Architecture 7
- 3.2 Text Preprocessing Pipeline 8
- 3.3 Feature Extraction via TF-IDF Vectorization 9
- 3.4 Theoretical Formulation of Classification Models 9
- 3.5 Risk Score via Sigmoid Mapping 10

4. Implementation & Experimental Results 10
- 4.1 Experimental Setup & Evaluation Metrics 10
- 4.2 Quantitative Model Comparison 11
- 4.3 Confusion Matrix Breakdown & Error Analysis 11

5. System Verification & Live Interface Testing 12
- 5.1 Detailed Analysis of Live Test Cases 13

6. Challenges Encountered & Engineering Solutions 14

7. Ethical Considerations, Applications & Future Scope 14
- 7.1 Data Privacy and Ethical AI Considerations 14
- 7.2 Practical Real-World Applications 14
- 7.3 Future Scope and Enhancements 15

8. Conclusion 15

9. References 15

Appendix: Complete Source Code 16
- A.1 Phase 1: Data Preprocessing & EDA (src/preprocess.py) 16
- A.2 Phase 2: Model Training & Evaluation (src/train_evaluate.py) 17
- A.3 Phase 3: Streamlit Web Application (app.py) 19

Abstract

Mobile Short Message Service (SMS) remains one of the most ubiquitous communication channels globally, making it a primary target for unsolicited commercial advertisements (spam) and malicious social engineering attacks (smishing, lottery fraud, and financial scams). Traditional rule-based keyword blacklists fail to adapt to evolving adversarial phrasing and shortcode obfuscation. This report presents **SpamShield**, an end-to-end Artificial Intelligence text classification system engineered to automatically distinguish between legitimate conversational messages (`ham`) and fraudulent or unsolicited messages (`spam`).

Using the benchmark UCI SMS Spam Collection dataset of 5,572 raw records (5,169 unique messages after deduplication), we implement a modular Python pipeline incorporating regular-expression text normalization, stop-word filtering, and Term Frequency–Inverse Document Frequency (TF-IDF) unigram and bigram feature extraction (3,000 features). We evaluate three supervised learning paradigms: Multinomial Naive Bayes, Logistic Regression, and Linear Support Vector Machine (LinearSVC). Evaluation on a stratified 80/20 train-test split shows that **Linear SVM** achieves the best overall performance, with an **Accuracy of 98.07%**, **Precision of 97.44%**, **Recall of 87.02%**, and **F1-Score of 91.94%**. Finally, the best model is deployed in an interactive **Streamlit** web application that maps the raw hyperplane decision distance through a logistic sigmoid function to produce an interpretable, real-time **Risk Score (0%–100%)** for end users.

1. Introduction & Problem Identification

1.1 Background and Context

In the modern digital ecosystem, mobile devices serve as the primary gateway for personal communication, two-factor authentication (2FA), banking alerts, and academic coordination. Because SMS messages enjoy an exceptionally high open rate—often exceeding 95% within minutes of delivery—cybercriminals and aggressive marketers exploit this medium to distribute unsolicited bulk messages (spam) and fraudulent phishing lures (smishing). Unlike email ecosystems, which benefit from decades of server-side Sender Policy Framework (SPF), DomainKeys Identified Mail (DKIM), and multi-layered gateway filtering, SMS infrastructure historically lacked built-in sender verification, allowing attackers to spoof alphanumeric sender IDs or rotate disposable SIM cards effortlessly.

1.2 Problem Statement

Given an arbitrary, unstructured text message M consisting of a sequence of characters, symbols, and colloquial abbreviations, the computational problem is to learn a mapping function f: $\square \rightarrow \square$, where $\square \subset \mathbb{R}^d$ is a d-dimensional numerical feature representation of the text and $\square = \{0, 1\}$ is the binary classification target (0 = Ham, 1 = Spam). Furthermore, to make the system actionable for non-technical users, the model must output a continuous probability or risk estimate $P(Y = 1 \mid X) \in [0, 1]$ alongside the discrete class prediction.

Formal Input and Output Specification:

- **System Input:** A raw, unstructured string representing an SMS text message (from 2 to 910 characters in length, containing alphanumeric tokens, punctuation, currency symbols, and numbers).

- **Intermediate Representation:** A sparse, L_2-normalized TF-IDF vector $\mathbf{x} \in \mathbb{R}^{3000}$ capturing unigram and bigram lexical importance.
- **System Output (1) – Binary Classification Verdict:** $\hat{y} \in \{0$ (Safe: Legitimate/Ham), 1 (Threat Detected: Spam/Scam)$\}$.
- **System Output (2) – Risk Score:** A continuous percentage $R \in [0\%, 100\%]$ quantifying threat confidence.

1.3 Objectives

1. **Data Curation & Preprocessing:** Cleanse the raw UCI SMS Spam Collection corpus by removing structural CSV artifacts, eliminating duplicate entries, normalizing casing, and stripping non-alphanumeric noise.
2. **Exploratory Data Analysis (EDA):** Quantify structural disparities between legitimate and spam messages across character length, word count, class balance, and high-frequency lexical triggers.
3. **Feature Engineering:** Construct a robust N-gram TF-IDF vectorization pipeline capable of capturing both individual trigger words (e.g., `free`, `txt`, `claim`) and two-word contextual phrases.
4. **Comparative Model Evaluation:** Train and benchmark three distinct AI classification families—probabilistic (Multinomial Naive Bayes), linear-probabilistic (Logistic Regression), and maximum-margin (Linear SVM)—using stratified validation metrics.
5. **Actionable Deployment:** Build and test a user-friendly Streamlit dashboard that delivers instantaneous predictions and risk scores on real-world and adversarial text inputs.

1.4 Real-World Significance & Cyber Security Relevance

For students specializing in **Cyber Security**, SMS spam detection is a foundational defense against **Social Engineering** and **Credential Harvesting**. Financial fraud in developing and developed mobile markets frequently begins with an urgent SMS claiming that a bank account is suspended or that the user has won a lottery prize. By automating threat detection at the application layer and educating users via an explicit Risk Score, **SpamShield** directly mitigates financial loss, identity theft, and malware propagation.

2. Dataset Description & Exploratory Data Analysis

2.1 Data Source and Structure

This project utilizes the **UCI SMS Spam Collection Dataset** (sourced via Kaggle), a publicly available benchmark corpus compiled by Tiago A. Almeida and José María Gómez Hidalgo. The raw CSV file (`spam.csv`, encoded in `latin-1`) initially contains 5,572 rows and 5 columns:

- `v1`: The ground-truth class label (`ham` or `spam`).
- `v2`: The raw SMS text string.
- `Unnamed: 2`, `Unnamed: 3`, `Unnamed: 4`: Spurious overflow columns caused by unescaped commas in a small fraction of raw text messages (50, 12, and 6 non-null entries, respectively).

2.2 Data Cleaning and Deduplication

During Phase 1 (`src/preprocess.py`), we executed strict data hygiene protocols:

1. **Column Pruning & Renaming:** Dropped the three `Unnamed` overflow columns and standardized schema headers to `label` and `message`.
2. **Null & Duplicate Removal:** Checked for null records and identified **403 exact duplicate messages** (frequently caused by repeated automated carrier alerts or identical spam campaigns). Removing these duplicates yielded a clean, leak-free corpus of **5,169 unique SMS records**.
3. **Target Encoding:** Mapped the categorical labels into a binary numerical target vector $y \in \{0, 1\}$, where ham $\mapsto 0$ and spam $\mapsto 1$.

2.3 Class Imbalance Analysis

A critical requirement in classification problems is evaluating target distribution skewness. Across the 5,169 unique messages:

- **Legitimate (`ham`):** 4,516 messages (87.37%)
- **Fraudulent/Unsolicited (`spam`):** 653 messages (12.63%)

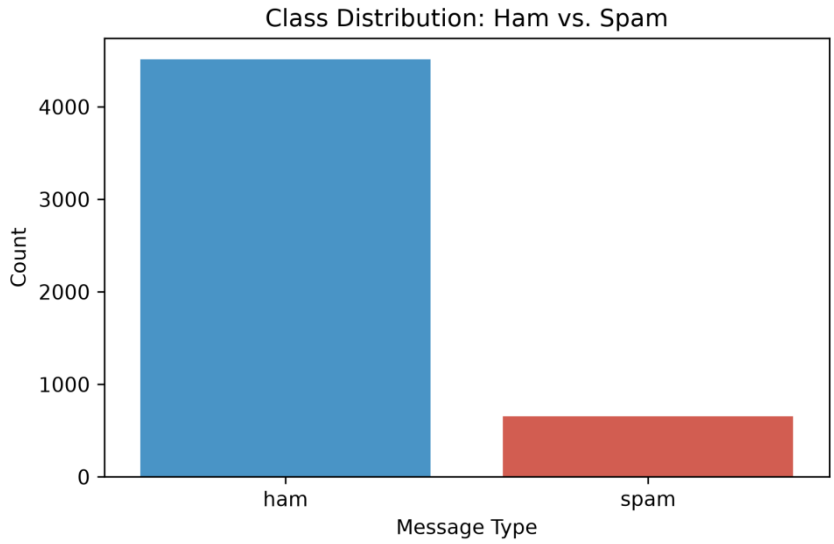


Figure 2.1: *Class distribution of ham and spam messages (5,169 records).*

Implication of Class Imbalance: The dataset exhibits an approximate 7:1 ratio in favor of legitimate messages. If a naive classifier simply predicted `ham` for every input, it would achieve a misleading 87.37% accuracy while failing to catch a single scam message (0% recall). Consequently, our methodology mandates **stratified train-test splitting** and prioritizes **Precision, Recall, and F1-Score** over raw accuracy.

2.4 Statistical Summary of Structural Features

To uncover structural patterns prior to vectorization, we engineered two meta-features: `char_length` (total characters per message) and `word_count` (whitespace-delimited tokens per message).

Class	Feature	Count (N)	Mean (μ)	Median	Min	Max	Std. Dev. (σ)
Ham (0)	Character Length	4,516	70.46	52.0	2	910	56.36
Spam (1)	Character Length	653	137.89	149.0	13	224	30.14
Ham (0)	Word Count	4,516	14.13	11.0	1	171	11.12
Spam (1)	Word Count	653	23.68	25.0	2	35	5.97

Table 2.1: *Descriptive statistics of structural features by class.*

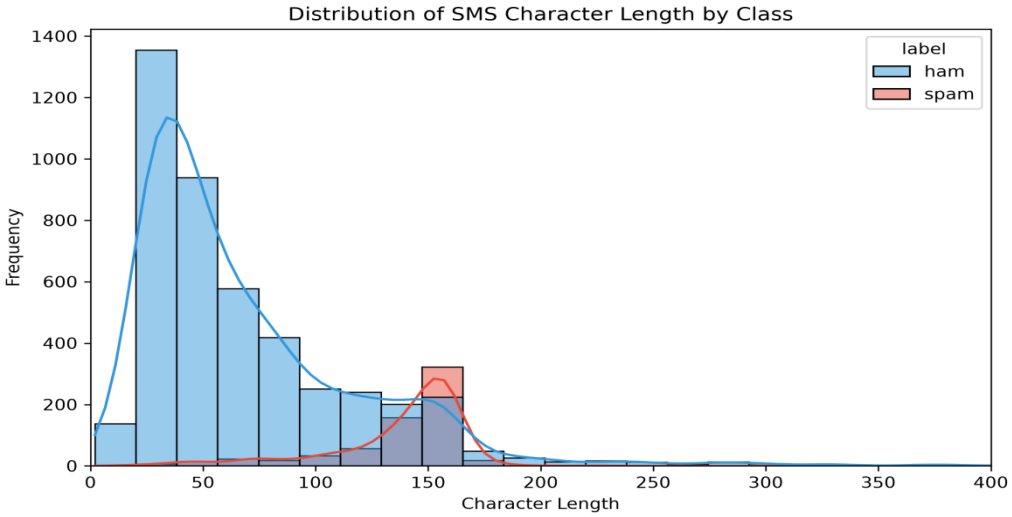


Figure 2.2: *Distribution of SMS character length by class (histogram with KDE).*

Key Analytical Insights from EDA:

1. **Character Length Separation:** Legitimate `ham` messages are predominantly brief conversational exchanges, averaging 70.46 characters with a median of 52.0. Conversely, `spam` messages average **137.89 characters** (median 149.0) with a much tighter standard deviation (σ = 30.14), as visible in Figure 2.2. Spammers pack as much promotional copy, prize claims, and shortcode instructions as possible right up to the standard 160-character GSM 03.38 single-SMS limit.
2. **Lexical Frequency Divergence:** By stripping standard English stop words and analyzing token frequencies (Figure 2.3), we observe a clear vocabulary boundary, summarized in Table 2.2.

Table 2.2: *Top five tokens per class after stop-word removal.*

Rank	Top Spam Tokens	Frequency	Top Ham Tokens	Frequency
1	`free`	188	`just`	274
2	`txt`	126	`ltgt`	254
3	`mobile`	105	`dont`	253
4	`text`	103	`got`	225
5	`stop`	103	`know`	221

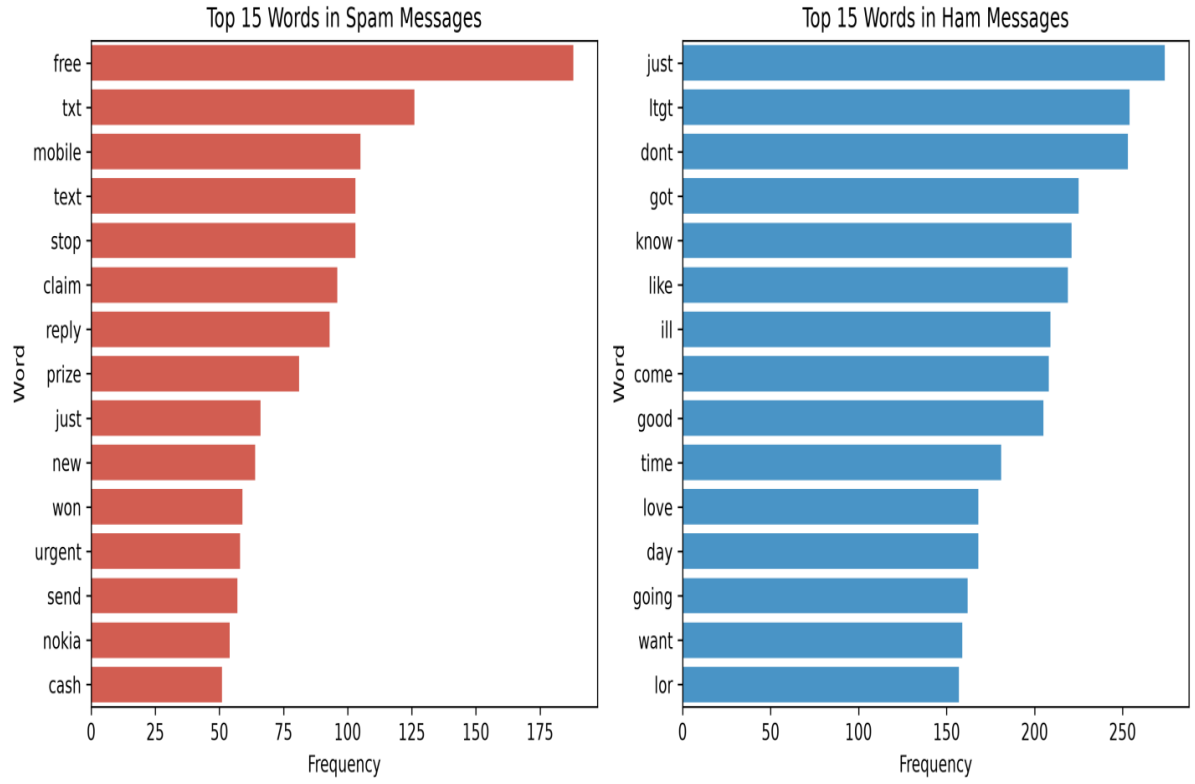


Figure 2.3: *Top 15 words in spam messages (left) and ham messages (right).*

Spam messages rely heavily on financial incentives (`free`, `prize`, `cash`, `won`), urgency triggers (`urgent`, `claim`), and call-to-action verbs (`txt`, `text`, `reply`, `stop`). Legitimate messages revolve around personal verbs, informal contractions (`dont`, `ill`), and anonymized numerical placeholders (`ltgt`, representing `<#>` in the original corpus).

3. Methodology & System Architecture

3.1 System Architecture

The **SpamShield** solution is architected into three decoupled tiers: the **Data & Preprocessing Pipeline** (`src/preprocess.py`), the **Model Training & Evaluation Engine** (`src/train_evaluate.py`), and the **Real-Time Inference & Risk Scoring Web App** (`app.py`). Figure 3.1 shows the data flow between them.

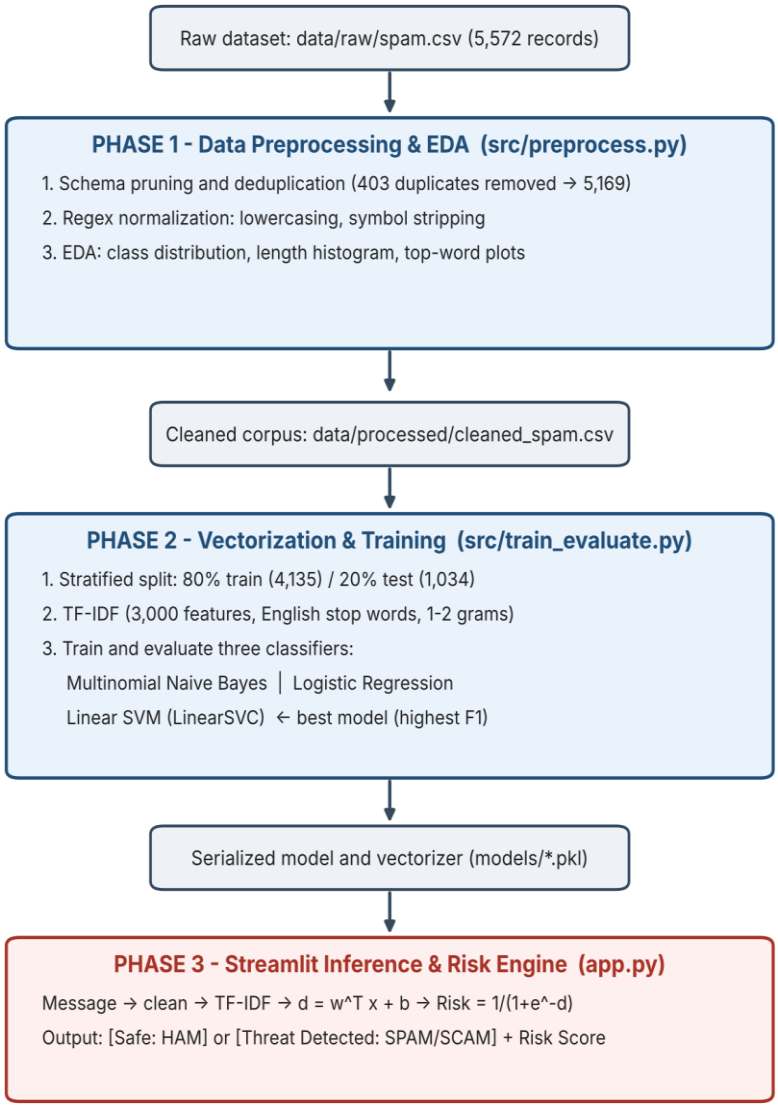


Figure 3.1: *SpamShield system architecture.*

3.2 Text Preprocessing Pipeline

Raw SMS text is notoriously noisy. In `clean_text()`, each document undergoes deterministic transformations:

1. **Case Folding:** All characters are converted to lowercase so that "URGENT!", "Urgent", and "urgent" map to the identical lexical token.
2. **Regular Expression Filtering:** The pattern `re.sub(r'[^a-z0-9\s]', '', text)` strips all punctuation, currency signs (£, $), and special symbols while preserving alphanumeric characters (a–z, 0–9), ensuring that shortcode numbers (e.g., `87121`, `80086`) remain intact as strong spam indicators.
3. **Whitespace Collapse & Null Handling:** Consecutive spaces are collapsed via `re.sub(r'\s+', ' ', text).strip()`, and messages consisting purely of punctuation are safely coerced via `.fillna('')` to prevent `NaN` decoding exceptions in `scikit-learn`.

3.3 Feature Extraction via TF-IDF Vectorization

Machine learning algorithms cannot operate directly on raw strings. Rather than using simple Bag-of-Words (BoW) raw counts—which over-weight common words—we apply **Term Frequency–Inverse Document Frequency (TF-IDF)** using `TfidfVectorizer(max_features=3000, stop_words='english', ngram_range=(1, 2))`. For a term (unigram or bigram) t in document d within a training corpus of N documents:

$$\mathrm{TF{-}IDF}(t, d) = \mathrm{TF}(t, d) \times \mathrm{IDF}(t)$$

where TF(t, d) is the frequency of term t in document d, and the Inverse Document Frequency with `scikit-learn` smoothing is computed as:

$$\mathrm{IDF}(t) = \ln\left(\frac{1+N}{1+\mathrm{df}(t)}\right) + 1$$

Here, df(t) is the number of documents containing term t. Finally, each document vector $\mathbf{x}$ is normalized to unit Euclidean (L_2) length:

$$\mathbf{x}_{\text{norm}} = \frac{\mathbf{x}}{\|\mathbf{x}\|_2} = \frac{\mathbf{x}}{\sqrt{\sum_{j=1}^{3000} x_j^2}}$$

Setting `ngram_range=(1, 2)` allows the feature space to capture both single words (`free`, `claim`) and two-word compounds (`cash prize`, `txt claim`, `free entry`), yielding sparse matrices of shape (4135, 3000) for training and (1034, 3000) for testing.

3.4 Theoretical Formulation of Classification Models

3.4.1 Multinomial Naive Bayes (MultinomialNB)

Multinomial Naive Bayes is a probabilistic classifier based on Bayes' theorem with a conditional-independence assumption between features given the class label $y \in \{0, 1\}$:

$$P(y \mid \mathbf{x}) \propto P(y) \prod_{j=1}^{d} P(x_j \mid y)^{x_j}$$

With Laplace (additive) smoothing parameter $\alpha = 1.0$, the likelihood of feature j given class y is estimated as:

$$\hat{P}(x_j \mid y) = \frac{N_{yj} + \alpha}{\sum_{k=1}^{d} N_{yk} + \alpha d}$$

While computationally fast and highly effective at achieving high precision, Naive Bayes assumes token independence, which can degrade recall when spam messages use complex phrasing.

3.4.2 Logistic Regression (LogisticRegression)

Logistic Regression models the posterior probability of the spam class directly, applying the logistic sigmoid function to a linear combination of TF-IDF features:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T\mathbf{x} + b)}}$$

The weight vector $\mathbf{w}$ and bias b are optimized by minimizing the L_2-regularized binary cross-entropy (log-loss) objective over the training set. Because of the 87.4% vs. 12.6% class imbalance and strong

L_2 shrinkage on sparse TF-IDF vectors, standard Logistic Regression tends to be conservative—yielding high precision (97.78%) but lower recall (67.18%) at the default 0.5 threshold.

3.4.3 Linear Support Vector Machine (LinearSVC)

Linear Support Vector Classification seeks the maximum-margin hyperplane $\mathbf{w}^T\mathbf{x} + b = 0$ that separates `ham` ($y_i = -1$) and `spam` ($y_i = +1$) in the 3,000-dimensional TF-IDF space. It solves the unconstrained primal optimization problem using the squared hinge loss:

$$\min_{\mathbf{w}, b} \ \tfrac{1}{2}\|\mathbf{w}\|_2^2 + C \sum_{i=1}^{n} \max\left(0,\ 1 - y_i(\mathbf{w}^T\mathbf{x}_i + b)\right)^2$$

High-dimensional text classification problems are frequently linearly separable. Because `LinearSVC` focuses directly on the support vectors near the decision boundary rather than on global class prior probabilities, it is more resilient to moderate class imbalance than uncalibrated Logistic Regression, delivering superior recall and F1-Score.

3.5 Risk Score via Sigmoid Mapping

Unlike `LogisticRegression`, `LinearSVC` outputs a continuous, signed functional distance $d(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b \in (-\infty, +\infty)$ from the separating hyperplane rather than a probability. To present an intuitive **Risk Score** in the Streamlit app (`app.py`), we pass the decision-function output through a logistic sigmoid transformation:

$$\text{Risk Score}(\mathbf{x}) = \left(\frac{1}{1 + e^{-d(\mathbf{x})}}\right) \times 100\%$$

- When $d(\mathbf{x}) = 0$ (exactly on the decision boundary), Risk Score = 50.00%.
- When $d(\mathbf{x}) > 0$ (classified as **SPAM**), Risk Score $\in$ (50%, 100%).
- When $d(\mathbf{x}) < 0$ (classified as **HAM**), Risk Score $\in$ (0%, 50%).

The sigmoid is a monotonic rescaling of the margin rather than a statistically calibrated probability (which would require, for example, Platt scaling fitted on held-out data); it is used here as an interpretable confidence indicator.

4. Implementation & Experimental Results

4.1 Experimental Setup & Evaluation Metrics

The cleaned dataset (5,169 messages) was partitioned using `train_test_split(..., test_size=0.20, random_state=42, stratify=y)`, resulting in:

- **Training Set (80%):** 4,135 messages (3,613 Ham, 522 Spam)
- **Testing Set (20%):** 1,034 messages (903 Ham, 131 Spam)

To evaluate performance under class imbalance, four standard metrics were computed from True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN):

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \qquad \text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \qquad \text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

4.2 Quantitative Model Comparison

Table 4.1 presents the empirical results generated by `src/train_evaluate.py` and logged to `outputs/metrics_report.txt`.

Table 4.1: *Performance comparison of the three classifiers on the 1,034-message test set.*

Classification Model	Accuracy	Precision	Recall	F1-Score	Rank
Multinomial Naive Bayes	96.91% (0.969052)	**98.06%** (0.980583)	77.10% (0.770992)	86.32% (0.863248)	2nd
Logistic Regression	95.65% (0.956480)	97.78% (0.977778)	67.18% (0.671756)	79.64% (0.796380)	3rd
Linear SVM (`LinearSVC`)	**98.07%** (0.980658)	97.44% (0.974359)	**87.02%** (0.870229)	**91.94%** (0.919355)	**1st**

4.3 Confusion Matrix Breakdown & Error Analysis

The confusion matrices over the 1,034 unseen test messages (903 actual Ham, 131 actual Spam), shown in Figure 4.1 and tabulated in Table 4.2, reveal the trade-off between false alarms (FP) and missed threats (FN).

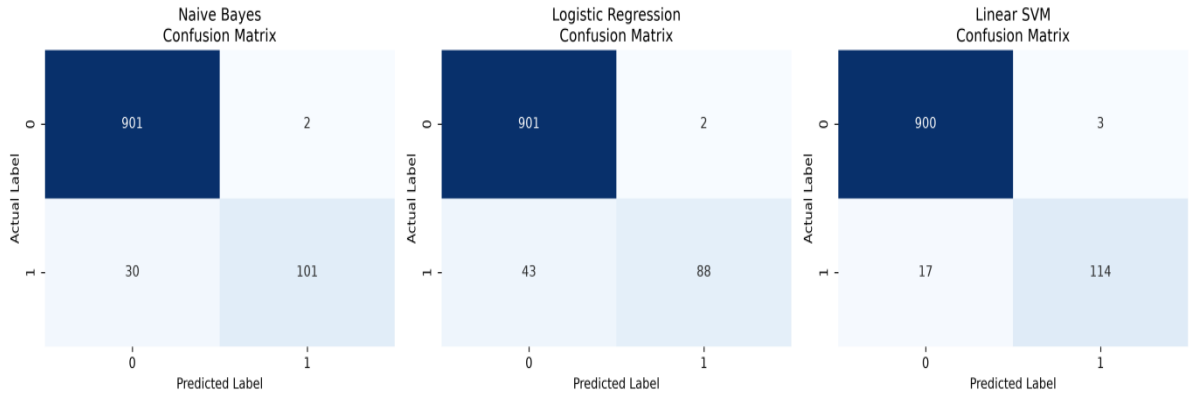


Figure 4.1: *Confusion matrices for Naive Bayes, Logistic Regression, and Linear SVM.*

Table 4.2: *Confusion matrix counts per model (test set).*

Model	TN (Ham → Ham)	FP (Ham → Spam)	FN (Spam → Ham)	TP (Spam → Spam)
Naive Bayes	901	2	30	101
Logistic Regression	901	2	43	88
Linear SVM	900	3	**17**	**114**

Interpretation of Results:

1. **Why Linear SVM won:** `LinearSVC` achieved the highest **F1-Score (91.94%)** and **Accuracy (98.07%)**. Crucially, it detected **114 of 131 spam messages (87.02% recall)**—catching 13 more scams than Naive Bayes and 26 more than Logistic Regression—while misclassifying only 3 of 903 legitimate messages (0.33% false-positive rate).
2. **Low false-positive rate across all models:** In SMS filtering, blocking a legitimate message (FP) is highly disruptive to the user. All three models kept FP ≤ 3 out of 903 legitimate messages, confirming that the 3,000-feature unigram/bigram TF-IDF representation cleanly isolates legitimate conversational vocabulary.

5. System Verification & Live Interface Testing

To validate the deployed Streamlit interface (`app.py`), we conducted five structured live test cases spanning overt financial scams, subscription spam, everyday university conversations, and an adversarial borderline input. Table 5.1 summarizes the inputs, verdicts, and risk scores; the corresponding screenshots follow in Figures 5.1–5.5.

Table 5.1: *Summary of live verification test cases.*

ID	Scenario	Input SMS Message	Predicted Verdict	Risk Score
TC-01	Prize / Lottery Scam	"come there as soon as possibleURGENT! You have won a FREE £1000 cash prize and a new Nokia mobile! Txt CLAIM to 87121 now!"	**SPAM / SCAM**	**91.26%**
TC-02	Subscription / Shortcode Spam	"FREE entry into our £250 weekly comp! Text WIN to 80086 now to claim your guaranteed gift voucher. T&Cs apply."	**SPAM / SCAM**	**84.36%**
TC-03	Academic / University Ham	"Hey bro, are we still meeting at the CSIT lab at 11 AM tomorrow to finalize the PAI project report?"	**LEGITIMATE (HAM)**	**23.21%**
TC-04	Everyday Conversational Ham	"I'm running about 10 minutes late, traffic near the main gate is packed. Don't wait up for me."	**LEGITIMATE (HAM)**	**28.75%**
TC-05	Adversarial / Borderline Trigger	"Call me back when you are free, I have some urgent news to tell you about the cash we collected."	**SPAM / SCAM** (false-positive edge case)	**61.00%**

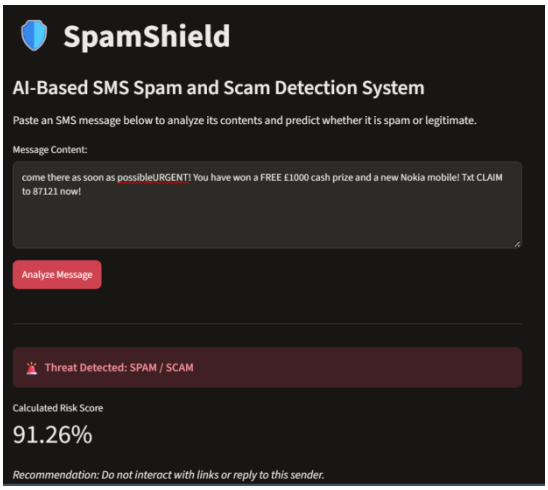


Figure 5.1: *TC-01 – Prize / lottery scam (91.26%).*

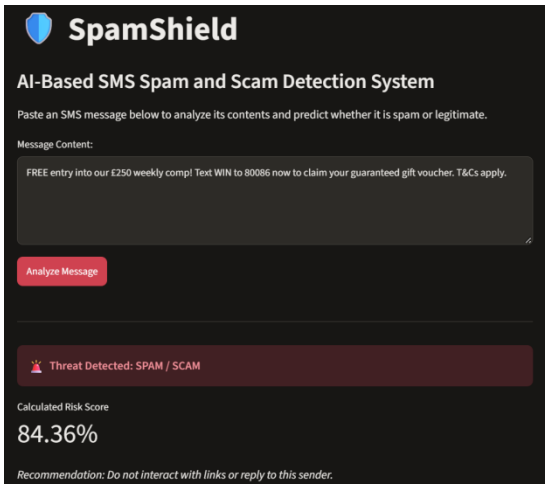


Figure 5.2: *TC-02 – Subscription spam (84.36%).*

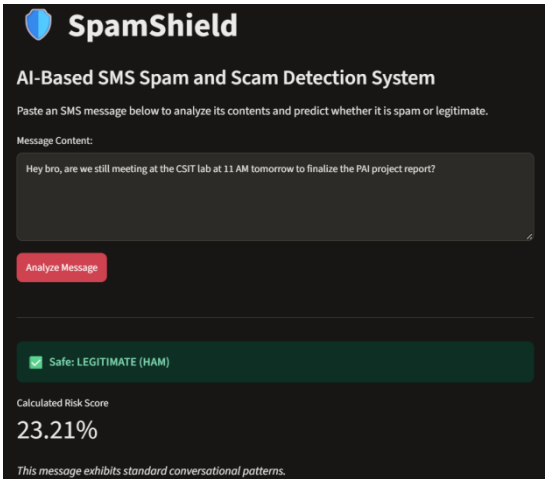


Figure 5.3: *TC-03 – Academic ham (23.21%).*

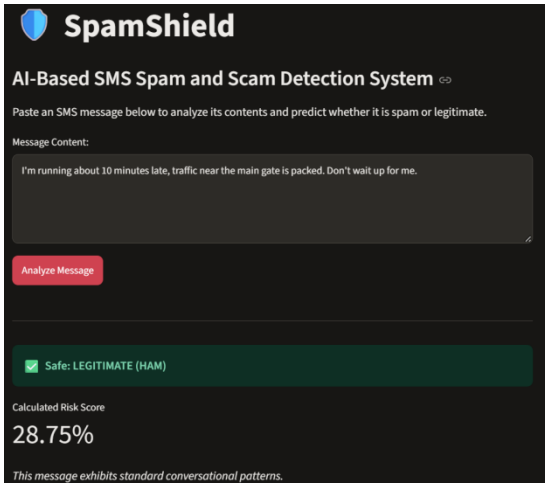


Figure 5.4: *TC-04 – Everyday ham (28.75%).*

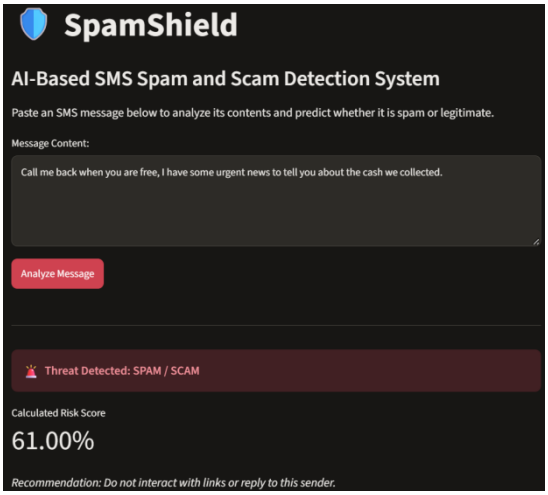


Figure 5.5: *TC-05 – Borderline trigger (61.00%).*

5.1 Detailed Analysis of Live Test Cases

- **Resilience to conversational prefix injection (TC-01):** In Figure 5.1, the conversational phrase "come there as soon as possible" was prepended directly onto the scam message (`possibleURGENT!`). Despite this obfuscation, the high TF-IDF weights of `free`, `1000`, `cash prize`, `nokia mobile`, and `txt claim` pushed the SVM hyperplane distance strongly positive, resulting in a decisive **91.26% Risk Score**.
- **Accurate safe classification (TC-03 & TC-04):** Normal university coordination (`CSIT lab`, `PAI project report`) and traffic updates (`10 minutes late`, `main gate`) contain no promotional bigrams, yielding low risk scores of **23.21%** and **28.75%**, respectively.
- **Edge-case / adversarial behavior (TC-05):** In Figure 5.5, a benign sentence was intentionally crafted using three of the top 15 spam words (`free`, `urgent`, `cash`). Because TF-IDF + LinearSVC evaluates lexical frequency rather than deep sequential semantics, it flags the message as spam, **but with a moderate 61.00% Risk Score** (compared with 84%–91% for real scams). This demonstrates why displaying a continuous Risk Score is valuable: a user seeing 61.00% knows the message sits close to the 50% decision threshold and warrants a quick visual check rather than immediate deletion.

6. Challenges Encountered & Engineering Solutions

During the development of **SpamShield**, the team encountered and resolved several technical and algorithmic challenges:

1. **Deprecated package naming & environment pathing.** *Challenge:* `pip install sklearn` failed due to PyPI deprecation, and launching `streamlit run app.py` on Windows with Python 3.14 raised a `CommandNotFoundException` because the user `Scripts` folder was missing from the system `PATH`. Running `python app.py` directly also caused `@st.cache_resource` to return `NoneType`. *Solution:* Standardized dependencies in `requirements.txt` using `scikit-learn` and invoked the Streamlit runtime via module execution (`python -m streamlit run app.py`).
2. **Pandas 3.0+ strict NaN string coercion.** *Challenge:* Certain raw SMS messages consisted solely of punctuation or emoticons (e.g., `:-)`) and became empty strings after regex cleaning. When exported to `cleaned_spam.csv` and reloaded in Pandas 3.0+, `.astype(str)` preserved `np.nan` objects, causing `TfidfVectorizer` to crash with `ValueError: np.nan is an invalid document`. *Solution:* Chained `.fillna('').astype(str)` before vectorization in `src/train_evaluate.py` to guarantee valid strings across all 5,169 rows.
3. **Probability estimates for maximum-margin classifiers.** *Challenge:* `LinearSVC` outperformed Naive Bayes and Logistic Regression in F1-Score (91.94%) but lacks a native `predict_proba()` method in `scikit-learn`. *Solution:* Extracted the raw signed distance from the separating hyperplane via `model.decision_function()` and applied the sigmoid transformation $1 / (1 + e^{-d})$ to produce a smooth, interpretable Risk Score (0%–100%).
4. **Semantic blindness of bag-of-words / n-grams.** *Challenge:* As shown in test case TC-05, linear TF-IDF models cannot distinguish between "you are free" (availability) and "FREE entry" (cost), leading to borderline false positives when multiple trigger tokens co-occur in casual speech.

7. Ethical Considerations, Applications & Future Scope

7.1 Data Privacy and Ethical AI Considerations

- **Local on-device inference:** SMS messages contain highly sensitive personally identifiable information (PII), banking OTPs, and private conversations. **SpamShield** executes 100% of its text preprocessing and SVM inference locally, without transmitting user text to external third-party cloud APIs.
- **Mitigating false-positive censorship:** Automatically deleting messages classified as spam poses an ethical and operational risk if an urgent personal message (like TC-05) is misclassified. By designing SpamShield as a **decision-support warning system** that outputs a Risk Score and safety recommendation rather than silently deleting messages, user autonomy is preserved.

7.2 Practical Real-World Applications

1. **Telecom carrier SMS gateways:** Deploying the lightweight `LinearSVC` model (inference in under 2 milliseconds) at the Short Message Service Center (SMSC) level to flag bulk smishing campaigns before they reach subscribers.

2. **Mobile OS messaging wrapper:** Integrating the model via ONNX or TensorFlow Lite into Android/iOS messaging clients to display a real-time risk banner on incoming texts from unknown numbers.
3. **Enterprise helpdesk & WhatsApp Business filtering:** Filtering automated scam inquiries and promotional bot messages in customer-support queues.

7.3 Future Scope and Enhancements

- **Contextual deep-learning embeddings:** Upgrading the TF-IDF representation to pre-trained Transformer embeddings (such as DistilBERT or RoBERTa) to resolve polysemy (e.g., distinguishing "free time" from "free prize" in TC-05).
- **Multilingual & Roman Urdu support:** In Pakistan, a large share of SMS scams (such as fake Benazir Income Support Programme (BISP) or Easypaisa/JazzCash lottery lures) are written in Roman Urdu or Urdu script. Expanding the training data with local Pakistani telecom spam corpora would make SpamShield directly deployable for national cyber security defense.
- **URL & phone-number reputation integration:** Combining text classification with real-time VirusTotal or Google Safe Browsing API lookups whenever a hyperlink or phone number is detected inside the SMS body.

8. Conclusion

This Complex Computing Activity (CCA) designed, implemented, evaluated, and deployed **SpamShield**, an end-to-end AI solution for SMS spam and scam detection. By rigorously preprocessing the UCI SMS Spam Collection dataset, conducting statistical and visual exploratory analysis, and benchmarking three machine learning algorithms over 3,000 TF-IDF unigram and bigram features, we demonstrated that **Linear SVM (`LinearSVC`)** provides the best balance between threat detection (**87.02% Recall**) and precision (**97.44% Precision**, **91.94% F1-Score**). Coupled with an interactive Streamlit web application and a sigmoid-scaled Risk Score, SpamShield translates theoretical machine learning principles into a practical, interpretable defense tool against mobile social engineering.

9. References

1. Almeida, T. A., Gómez Hidalgo, J. M., & Yamakami, A. (2011). *Contributions to the study of SMS spam filtering: new collection and results*. Proceedings of the 11th ACM Symposium on Document Engineering (DocEng '11), Mountain View, CA, USA.
2. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
3. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
4. UCI Machine Learning Repository / Kaggle: *SMS Spam Collection Dataset*. https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset

Appendix: Complete Source Code

A.1 Phase 1: Data Preprocessing & EDA (src/preprocess.py)

```
import os
import re
from collections import Counter
import joblib
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.feature_extraction.text import TfidfVectorizer

os.makedirs('data/processed', exist_ok=True)
os.makedirs('models', exist_ok=True)
os.makedirs('outputs/figures', exist_ok=True)

def clean_text(text):
    """Lowercase and remove punctuation/special characters."""
    text = str(text).lower()
    text = re.sub(r'[^a-z0-9\s]', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

def main():
    raw_path = 'data/raw/spam.csv'
    if not os.path.exists(raw_path):
        raise FileNotFoundError(f"Could not find {raw_path}. Make sure spam.csv is in the
data/raw/ folder.")

    df = pd.read_csv(raw_path, encoding='latin-1')
    df = df[['v1', 'v2']].rename(columns={'v1': 'label', 'v2': 'message'})

    initial_count = len(df)
    df = df.dropna().drop_duplicates(keep='first').reset_index(drop=True)
    duplicates_removed = initial_count - len(df)

    df['char_length'] = df['message'].apply(len)
    df['word_count'] = df['message'].apply(lambda x: len(x.split()))
    df['cleaned_message'] = df['message'].apply(clean_text)
    df['target'] = df['label'].map({'ham': 0, 'spam': 1})

    stats = df.groupby('label')[['char_length', 'word_count']].agg(
        ['count', 'mean', 'median', 'min', 'max', 'std']
    )

    plt.figure(figsize=(6, 4))
    sns.countplot(data=df, x='label', hue='label', palette={'ham': '#3498db', 'spam': '#e74c3c'},
legend=False)
    plt.title('Class Distribution: Ham vs. Spam')
    plt.xlabel('Message Type')
    plt.ylabel('Count')
    plt.tight_layout()
    plt.savefig('outputs/figures/class_distribution.png', dpi=300)
    plt.close()

    plt.figure(figsize=(8, 5))
    sns.histplot(data=df, x='char_length', hue='label', bins=50, kde=True, palette={'ham':
'#3498db', 'spam': '#e74c3c'})
    plt.title('Distribution of SMS Character Length by Class')
    plt.xlabel('Character Length')
    plt.ylabel('Frequency')
    plt.xlim(0, 400)
```

```
    plt.tight_layout()
    plt.savefig('outputs/figures/message_length_dist.png', dpi=300)
    plt.close()

    tfidf_temp = TfidfVectorizer(stop_words='english')
    stop_words = tfidf_temp.get_stop_words()

    def get_top_words(series, n=15):
        words = ' '.join(series).split()
        words = [w for w in words if w not in stop_words and len(w) > 2]
        return Counter(words).most_common(n)

    top_spam = pd.DataFrame(get_top_words(df[df['label'] == 'spam']['cleaned_message']),
columns=['word', 'count'])
    top_ham = pd.DataFrame(get_top_words(df[df['label'] == 'ham']['cleaned_message']),
columns=['word', 'count'])

    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    sns.barplot(data=top_spam, x='count', y='word', ax=axes[0], color='#e74c3c')
    axes[0].set_title('Top 15 Words in Spam Messages')
    axes[0].set_xlabel('Frequency')
    axes[0].set_ylabel('Word')

    sns.barplot(data=top_ham, x='count', y='word', ax=axes[1], color='#3498db')
    axes[1].set_title('Top 15 Words in Ham Messages')
    axes[1].set_xlabel('Frequency')
    axes[1].set_ylabel('Word')

    plt.tight_layout()
    plt.savefig('outputs/figures/top_words_comparison.png', dpi=300)
    plt.close()

    tfidf = TfidfVectorizer(max_features=3000, stop_words='english', ngram_range=(1, 2))
    X_tfidf = tfidf.fit_transform(df['cleaned_message'])

    df.to_csv('data/processed/cleaned_spam.csv', index=False)
    joblib.dump(tfidf, 'models/tfidf_vectorizer.pkl')

    print("=== PHASE 1 COMPLETE ===")
    print(f"Initial records: {initial_count} | Duplicates removed: {duplicates_removed} | Final
records: {len(df)}")
    print("\n--- Statistical Summary ---")
    print(stats.to_string())
    print(f"\nTF-IDF Matrix Shape: {X_tfidf.shape}")

if __name__ == '__main__':
    main()
```

A.2 Phase 2: Model Training & Evaluation (src/train_evaluate.py)

```
import os
import joblib
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
```

```
os.makedirs('models', exist_ok=True)
os.makedirs('outputs/figures', exist_ok=True)

def main():
    processed_path = 'data/processed/cleaned_spam.csv'
    if not os.path.exists(processed_path):
        raise FileNotFoundError("Processed dataset not found! Please run 'python
src/preprocess.py' first.")

    df = pd.read_csv(processed_path)
    X_text = df['cleaned_message'].fillna('').astype(str)
    y = df['target']

    X_train_text, X_test_text, y_train, y_test = train_test_split(
        X_text, y, test_size=0.20, random_state=42, stratify=y
    )

    tfidf = TfidfVectorizer(max_features=3000, stop_words='english', ngram_range=(1, 2))
    X_train = tfidf.fit_transform(X_train_text)
    X_test = tfidf.transform(X_test_text)

    joblib.dump(tfidf, 'models/tfidf_vectorizer.pkl')

    models = {
        'Naive Bayes': MultinomialNB(),
        'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
        'Linear SVM': LinearSVC(random_state=42, dual='auto')
    }

    results = []
    trained_models = {}

    fig, axes = plt.subplots(1, 3, figsize=(15, 4))

    for idx, (name, model) in enumerate(models.items()):
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

        acc = accuracy_score(y_test, y_pred)
        prec = precision_score(y_test, y_pred)
        rec = recall_score(y_test, y_pred)
        f1 = f1_score(y_test, y_pred)

        results.append({
            'Model': name,
            'Accuracy': acc,
            'Precision': prec,
            'Recall': rec,
            'F1-Score': f1
        })
        trained_models[name] = model

        cm = confusion_matrix(y_test, y_pred)
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False, ax=axes[idx])
        axes[idx].set_title(f'{name}\nConfusion Matrix')
        axes[idx].set_xlabel('Predicted Label')
        axes[idx].set_ylabel('Actual Label')

    plt.tight_layout()
    plt.savefig('outputs/figures/confusion_matrices.png', dpi=300)
    plt.close()

    results_df = pd.DataFrame(results)
```

```
    report_text = f"=== MODEL EVALUATION REPORT ===\n\n{results_df.to_string(index=False)}\n"
    with open('outputs/metrics_report.txt', 'w') as f:
        f.write(report_text)

    best_model_name = results_df.sort_values(by='F1-Score', ascending=False).iloc[0]['Model']
    best_model = trained_models[best_model_name]
    joblib.dump(best_model, 'models/best_spam_model.pkl')

    print("=== PHASE 2 COMPLETE ===")
    print(results_df.to_string(index=False))

if __name__ == '__main__':
    main()
```

A.3 Phase 3: Streamlit Web Application (app.py)

```
import streamlit as st
import joblib
import re
import numpy as np

st.set_page_config(page_title="SpamShield", page_icon="□", layout="centered")

def clean_text(text):
    """Mirror the exact preprocessing steps from training."""
    text = str(text).lower()
    text = re.sub(r'[^a-z0-9\s]', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

@st.cache_resource
def load_models():
    """Load and cache the vectorizer and best model."""
    try:
        tfidf = joblib.load('models/tfidf_vectorizer.pkl')
        model = joblib.load('models/best_spam_model.pkl')
        return tfidf, model
    except FileNotFoundError:
        st.error("Model artifacts not found. Please ensure 'models/tfidf_vectorizer.pkl' and
'models/best_spam_model.pkl' exist.")
        st.stop()

tfidf, model = load_models()

st.title("□ SpamShield")
st.subheader("AI-Based SMS Spam and Scam Detection System")
st.markdown("Paste an SMS message below to analyze its contents and predict whether it is spam or
legitimate.")

user_input = st.text_area("Message Content:", height=150, placeholder="Enter text message
here...")

if st.button("Analyze Message", type="primary"):
    if user_input.strip() == "":
        st.warning("Please enter a message to analyze.")
    else:
        cleaned_input = clean_text(user_input)
        vectorized_input = tfidf.transform([cleaned_input])
        prediction = model.predict(vectorized_input)[0]

        if hasattr(model, "predict_proba"):
            risk_score = model.predict_proba(vectorized_input)[0][1]
        elif hasattr(model, "decision_function"):
            decision = model.decision_function(vectorized_input)[0]
```

```
            risk_score = 1 / (1 + np.exp(-decision))
        else:
            risk_score = float(prediction)

        risk_percentage = risk_score * 100
        st.divider()

        if prediction == 1:
            st.error("□ **Threat Detected: SPAM / SCAM**")
            st.metric(label="Calculated Risk Score", value=f"{risk_percentage:.2f}%")
            st.markdown("*Recommendation: Do not interact with links or reply to this sender.*")
        else:
            st.success("✅ **Safe: LEGITIMATE (HAM)**")
            st.metric(label="Calculated Risk Score", value=f"{risk_percentage:.2f}%")
            st.markdown("*This message exhibits standard conversational patterns.*")
```